point is likely to be high, making it more likely to be the one maximizing the sample.
- If a data point has a high predictive standard deviation (that is, uncertainty), then a random sample at that data point will also have a higher chance of having a high value, again, making the data point more likely to be chosen as the next point to query.

TS is likely to exploit with a random sample that is maximized in a region with high predictive mean, but the policy is just as likely to explore with another sample given the same situation. We see this in Figure 5.12, where one sample is maximized around -1.2, which has a relatively high predictive mean. If this is the sample we draw, then by evaluating the objective at -1.2, we will be *exploiting* the function. However, if we draw either of the other two samples, then we will be *exploring*, as the regions where the samples are maximized have high uncertainty.

This is quite an elegant tradeoff scheme. By using the randomness of the samples, TS directly leverages the probabilistic nature of our predictive model, the Gaussian process, to explore and exploit the search space. The randomized nature of TS means that at any given iteration of the BayesOpt loop, the policy might not make the best decision to trade off exploration and exploitation, but over time, in the aggregation of its decisions, the policy will be able to explore the space sufficiently and narrow down high-performing regions.

5.3.2 Implementation with BoTorch

Let's now move on to how to implement this TS policy in Python. Once again, the code is included in the `CH05/01 - BayesOpt loop.ipynb` notebook. Remember that for the BayesOpt policies we have seen, implementation comes down to declaring a BoTorch policy object and specifying the relevant information. Then, to find the data point that we should query next, we optimize the acquisition score as computed by the policy. This is, however, *not* the case with TS.

The main challenge in implementing TS as a generic PyTorch module that takes in a data point and scores it according to some criterion is that, sampling from a GP can only be done with a finite number of points. In the past, we have represented a sample from a GP using a sample of a high-dimensional multivariate Gaussian distribution of a dense grid over the search space. When samples of this dense Gaussian are plotted, they appear as though they are actual functions with smooth curves, but in reality, they are defined over a grid.

All of this is to say that it is computationally impossible to draw a sample as a function from a GP, since that would require an infinitely number of bits. A typical solution is to draw the corresponding multivariate Gaussian over a large number of points that span the input space, so that all regions in the search space are represented.

IMPORTANT **This is exactly what we will do to implement Thompson sampling: generate a large number of points throughout the search space and draw a sample from the multivariate Gaussian distribution corresponding to predictions from a Gaussian process over these points.**

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>